LAPORAN PRAKTIKUM
Algoritma dan Pemrograman Komputer

Disusun untuk Memenuhi Mata Kuliah Algoritma dan Pemrograman Komputer

Dosen: Jevri Tri Ardiansah, S.Pd., M.Eng., Ph.D.

Disusun Oleh:

Muhammad Zaki Aliyyudin 260535649406

PROGRAM STUDI S1 TEKNIK INFORMATIKA

DEPARTEMEN TEKNIK ELEKTRO DAN INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS NEGERI MALANG

2026

Daftar Isi

BAB I
PENDAHULUAN **3**
1.1 Latar Belakang 3
1.2 Rumusan Masalah 4
1.3 Tujuan4
1.4 Manfaat 4
BAB II
PEMBAHASAN **6**
2.1 Studi Kasus 1 6
2.2 Studi Kasus 2 9
2.3 Studi Kasus 3 11
2.4 Studi Kasus 4 12
2.5 Studi Kasus 5 12
2.6 Studi Kasus 6 14
2.7 Studi Kasus 7 16
2.8 Studi Kasus 8 17
2.9 Studi Kasus 9 17
2.10 Studi Kasus 10 18
BAB III
KESIMPULAN **19**
3.1 Kesimpulan 19

BAB I
PENDAHULUAN

1.1 Latar Belakang

Algoritma dan pemrograman merupakan dasar penting dalam memahami bagaimana suatu masalah dapat diselesaikan secara sistematis menggunakan komputer. Dalam proses pemrograman, tidak semua permasalahan dapat diselesaikan hanya dengan proses yang berjalan secara berurutan. Terdapat kondisi tertentu yang membutuhkan pengambilan keputusan berdasarkan data atau keadaan yang diberikan. Oleh karena itu, pemahaman mengenai struktur kontrol dan seleksi kondisi menjadi bagian penting dalam pembelajaran pemrograman, khususnya untuk menentukan tindakan yang harus dilakukan oleh program berdasarkan kondisi tertentu.

Pada praktikum ini, konsep struktur kontrol dan seleksi kondisi diterapkan menggunakan bahasa pemrograman C++. Praktikan mempelajari penggunaan struktur seleksi seperti if, if-else, dan if-else if untuk menentukan keluaran program berdasarkan kondisi yang terpenuhi. Selain itu, operator perbandingan dan operator logika digunakan untuk membuat serta menggabungkan kondisi dalam proses pengambilan keputusan. Penggunaan struktur seleksi tersebut memungkinkan program memberikan hasil yang berbeda sesuai dengan data yang dimasukkan oleh pengguna.

Melalui praktikum ini, konsep struktur kontrol dan seleksi kondisi tidak hanya dipelajari secara teori, tetapi juga diterapkan dalam beberapa permasalahan sederhana yang membutuhkan pengambilan keputusan, seperti menentukan kategori nilai, menghitung tarif berdasarkan jumlah tertentu, menentukan kategori kebutuhan cat, serta memberikan informasi berdasarkan kondisi dari data yang diinputkan. Dengan demikian, praktikan dapat memahami bagaimana struktur .

1.2 Rumusan Masalah

1. Studi Kelas 1
2. Studi Kelas 2
3. Studi Kelas 3
4. Studi Kelas 4
5. Studi Kelas 5
6. Studi Kasus 1
7. Studi Kasus 2
8. Studi Kasus 3
9. Studi Kasus 4
10. Studi Kasus 5

1.3 Tujuan

1. Memahami konsep dasar struktur perulangan dalam pemrograman C++.
2. Memahami penggunaan struktur perulangan `for`, `while`, dan `do-while` dalam program.
3. Memahami penggunaan kondisi dan operator perbandingan dalam menentukan proses perulangan.
4. Mampu menerapkan struktur perulangan untuk menyelesaikan permasalahan sederhana.
5. Mampu membuat program yang dapat melakukan proses secara berulang berdasarkan kondisi atau jumlah perulangan yang ditentukan.
6. Meningkatkan kemampuan dalam menyusun algoritma yang melibatkan proses perulangan dan pengulangan data.

1.4 Manfaat

1. Menambah pemahaman mengenai konsep struktur perulangan dalam pemrograman C++.
2. Meningkatkan kemampuan dalam menggunakan struktur `for`, `while`, dan `do-while` pada program.
3. Melatih kemampuan dalam menentukan kondisi dan batas perulangan sesuai kebutuhan program.
4. Membantu praktikan dalam menyelesaikan permasalahan pemrograman yang membutuhkan proses berulang.
5. Meningkatkan kemampuan berpikir logis dan sistematis dalam menyusun algoritma menggunakan perulangan.
6. Membantu praktikan membuat program yang dapat melakukan proses secara efisien tanpa menuliskan perintah yang sama secara berulang.

BAB II
PEMBAHASAN

2.1 Studi Kasus 1

Program :

```cpp
void calculateSquare(int number) {
  number *= number;
}
int main() {
   int num = 5;
   calculateSquare(num);
   cout << "Kuadrat dari " << num << " adalah " << num <<
endl;
   return 0;
}
```

Deskripsi :

`void calculateSquare(int number)` `{`	Membuat fungsi bernama `calculateSquare`. `void` berarti fungsi ini **tidak mengembalikan nilai**. `int number` adalah parameter untuk menerima angka yang akan dikuadratkan.
`number = number * number;`	Jika `number = 5`, maka: `number = 5 * 5` `number = 25;`

`int main() {`
`int num = 5;`
`calculateSquare(num);`
	Fungsi utama yang pertama kali dijalankan oleh program. Membuat variabel `num` bertipe `int`. Memanggil fungsi `calculateSquare`. Nilai `num`, yaitu 5, dikirim ke parameter `number` Nilainya adalah 5.

`cout << "Kuadrat dari " << num << " adalah " << num << endl;`	Menampilkan nilai `num`. Karena `num` masih `5`, hasilnya:

2.2 Studi Kasus 2

Program :

```C++
void recursion() {
  cout << "Halo." << endl;
  recursion();
}
int main() {
   recursion();
   return 0;
}
```

Deskripsi ;

`void recursion() {` `cout << "Halo." << endl;`	`Membuat fungsi bernama recursion.void berarti fungsi tidak mengembalikan nilai.Menampilkan tulisan:`
`recursion();` `cout << "Halo." << endl;`	`Fungsi recursion()` **`memanggil dirinya sendiri`**`.Setelah dipanggil, fungsi kembali menjalankan: Kemudian memanggil` `recursion()` `lagi.Begitu seterusnya.`

`int main() {`
`recursion();`
`return 0;`
	Fungsi utama program.
	Memanggil fungsi `recursion()` untuk pertama kali.
	Mengakhiri program.

2.3 Studi Kelas 3

Program :

```cpp
int sum(int n)
{

    if (n <= 0)
        return 0;

    return n + sum(n - 1);
}

int main()
{

    int n = 3;

    cout << "Jumlah dari 1 hingga " << n << " adalah: " <<
sum(n) << endl;

    return 0;
}
```

Deskripsi :

`int sum(int n)`	Membuat fungsi `sum` untuk menghitung jumlah angka dari `n` sampai 1
`if (n <= 0)`	Mengecek apakah nilai `n` sudah 0 atau kurang.
`return 0;`	Menghentikan perulangan fungsi dan mengembalikan nilai 0.

`return n + sum(n - 1);`	Menjumlahkan nilai `n` dengan hasil fungsi `sum` untuk angka sebelumnya. .
`int main()`	Menjadi bagian utama tempat program dijalankan.
`int n = 3;`	Menentukan nilai `n` sebesar 3.
`cout << ...`	Menampilkan hasil penjumlahan dari 1 sampai `n`.
`sum(n)`	Memanggil fungsi `sum` untuk menghitung hasil penjumlahan.
`return 0`	Menandakan program selesai dijalankan.

2.4 Studi Kelas 4

Program :

C++

```cpp
void printStars(int n)
{

    if (n <= 0)
        return;

    printStars(n - 1);

    for (int i = 0; i < n; i++)
    {

        cout << "* ";
    }

    cout << endl;
}

int main()
{

    int rows = 5;
```

```
    printStars(rows);

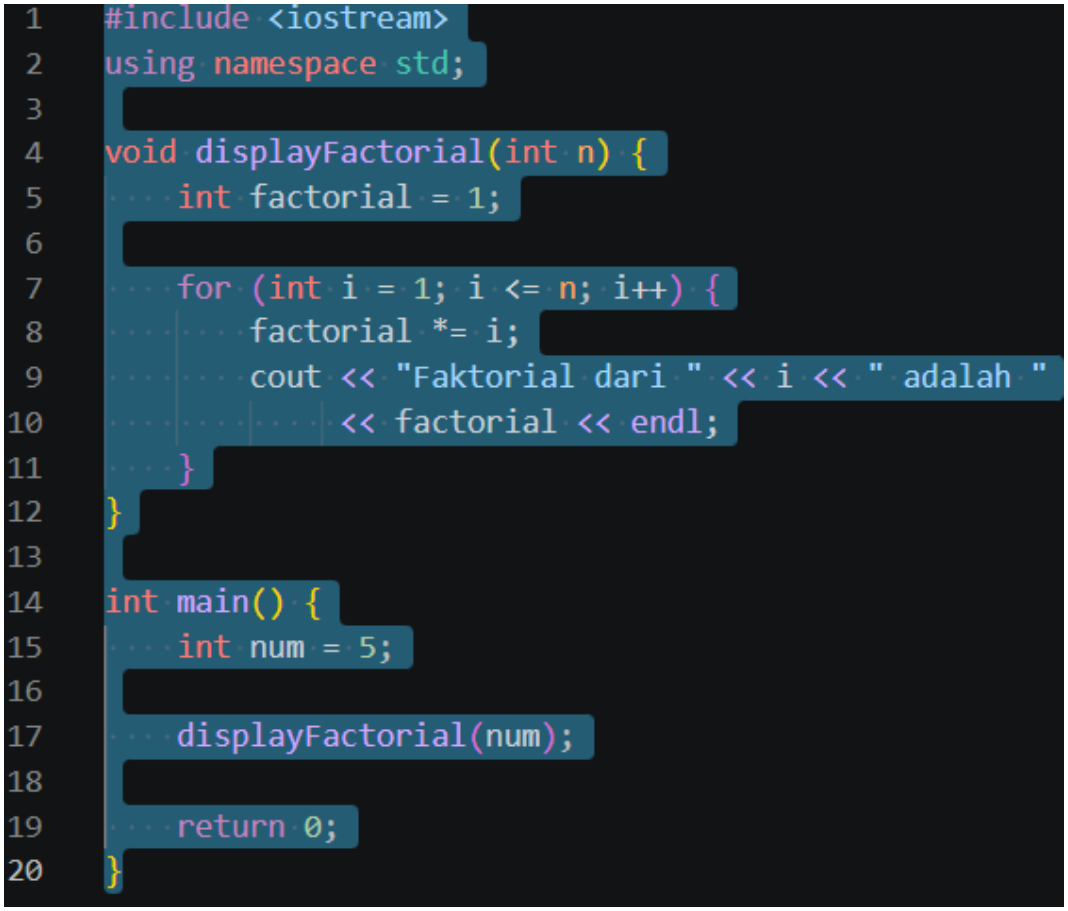
    return 0;
}
```

Deskripsi :

`void printStars(int n)`	Membuat fungsi printStars untuk menampilkan pola bintang.
`if (n <= 0)`	Mengecek apakah nilai `n` sudah 0 atau kurang.
`return;`	`Menghentikan fungsi jika n sudah 0 atau kurang.`
printStars(n - 1);	Memanggil fungsi kembali dengan nilai `n` yang dikurangi 1.
for (int i = 0; i < n; i++)	Mengulang pencetakan bintang sesuai nilai `n`.
cout << "* ";	Menampilkan satu bintang.
cout << endl;	Pindah ke baris berikutnya setelah bintang selesai dicetak.
int main()	Menjadi bagian utama tempat program dijalankan.
int rows = 5;	Menentukan jumlah baris pola bintang sebanyak 5.
printStars(rows);	Memanggil fungsi `printStars` dengan jumlah baris 5.
return 0;	Menandakan program selesai dijalankan.

2.5 Studi Kelas 5

Program :



```
#include <iostream>
using namespace std;

void displayFactorial(int n) {
    int factorial = 1;

    for (int i = 1; i <= n; i++) {
        factorial *= i;
        cout << "Faktorial dari " << i << " adalah "
             << factorial << endl;
    }
}

int main() {
    int num = 5;

    displayFactorial(num);

    return 0;
}
```

Deskripsi :

#include <iostream>	Digunakan untuk menggunakan fungsi input dan output seperti cout.
using namespace std;	Memungkinkan penggunaan fungsi dari library standar C++ tanpa harus menuliskan std::.
void displayFactorial(int n)	Membuat fungsi displayFactorial dengan parameter n bertipe int untuk menghitung dan menampilkan faktorial.
int factorial = 1;	Membuat variabel factorial dengan nilai awal 1 yang digunakan untuk menyimpan hasil perhitungan faktorial.
for (int i = 1; i <= n; i++)	Melakukan perulangan dari i = 1 sampai i sama

	dengan n untuk menghitung faktorial.
factorial *= i;	Mengalikan nilai factorial dengan nilai i pada setiap perulangan.
cout << "Faktorial dari " << i << " adalah "	Menampilkan teks yang menunjukkan nilai i dan hasil faktorialnya.
<< factorial << endl;	Menampilkan hasil faktorial dan kemudian berpindah ke baris berikutnya.
int main()	Menjadi fungsi utama tempat program mulai dijalankan.
int num = 5;	Membuat variabel num bertipe int dengan nilai 5 sebagai angka yang akan dihitung faktorialnya.
displayFactorial(num);	Menandakan bahwa program telah selesai dijalankan dengan normal.

2.6 Studi Kelas 6

Program :

```cpp
#include <iostream>
using namespace std;

int sumDigits(int n) {
    if (n == 0)
        return 0;

    return (n % 10) + sumDigits(n / 10);
}

int main() {
    int n = 222;

    cout << "Jumlah digit dari " << n << " adalah "
         << sumDigits(n) << endl;

    n = 333;

    cout << "Jumlah digit dari " << n << " adalah "
         << sumDigits(n) << endl;

    return 0;
}
```

`#include <iostream>`	Digunakan untuk menggunakan fungsi input dan output seperti `cout`.
`using namespace std;`	Memungkinkan penggunaan fungsi dari library standar C++ tanpa harus menuliskan `std::`.

`int sumDigits(int n)`	Membuat fungsi `sumDigits` dengan parameter `n` bertipe `int` untuk menghitung jumlah digit dari suatu bilangan.
`if (n == 0)`	Mengecek apakah nilai `n` sudah sama dengan 0 sebagai kondisi dasar dari fungsi rekursif.
`return 0;`	Mengembalikan nilai `0` jika nilai `n` sudah mencapai 0.
`return (n % 10) + sumDigits(n / 10);`	Mengambil digit terakhir menggunakan `n % 10`, kemudian menjumlahkannya dengan hasil pemanggilan fungsi `sumDigits` menggunakan `n / 10`.
`}`	Menandakan akhir dari fungsi `sumDigits`.
`int main()`	Menjadi fungsi utama tempat program mulai dijalankan.
`int n = 222;`	Membuat variabel `n` bertipe `int` dengan nilai awal `222`.
`cout << "Jumlah digit dari " << n << " adalah "`	Menampilkan teks yang berisi angka yang akan dihitung jumlah digitnya.
`<< sumDigits(n) << endl;`	Memanggil fungsi `sumDigits` untuk menghitung jumlah digit dari `n`, kemudian menampilkan hasilnya dan berpindah ke baris berikutnya.
`n = 333;`	Mengubah nilai variabel `n` menjadi `333` untuk melakukan perhitungan berikutnya.
`cout << "Jumlah digit dari " << n << " adalah "`	Menampilkan teks yang berisi angka `333` yang akan dihitung jumlah digitnya.
`<< sumDigits(n) << endl;`	Memanggil fungsi `sumDigits` untuk menghitung jumlah digit dari `333`, kemudian menampilkan hasilnya.

`return 0;`	Menandakan bahwa program telah selesai dijalankan dengan normal.
`}`	Menandakan akhir dari fungsi `main`.

BAB III
KESIMPULAN

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa pemahaman mengenai struktur perulangan merupakan bagian penting dalam pembuatan program menggunakan C++. Praktikan mempelajari penggunaan struktur perulangan seperti `for`, `while`, dan `do-while` untuk menjalankan suatu proses secara berulang berdasarkan kondisi tertentu. Selain itu, penggunaan kondisi dan variabel kontrol membantu dalam mengatur jumlah serta proses perulangan agar program dapat berjalan sesuai kebutuhan.

Melalui beberapa studi kasus yang telah dikerjakan, praktikan dapat menerapkan perulangan pada berbagai permasalahan sederhana, seperti menampilkan data secara berulang, melakukan perhitungan berulang, serta menjalankan program kembali sesuai pilihan pengguna. Praktikum ini juga melatih kemampuan berpikir logis dan sistematis dalam menentukan kondisi, batas perulangan, serta penggunaan struktur perulangan yang sesuai agar program dapat berjalan dengan baik dan menghasilkan keluaran yang diharapkan.

3.2 Link Github